

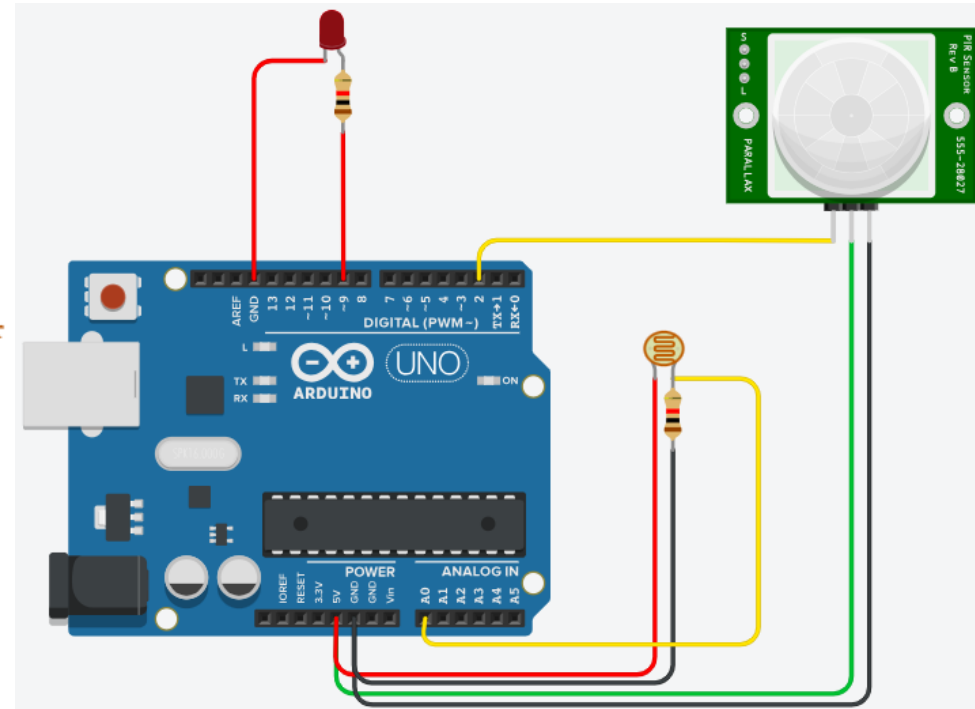
Module – 3:
Arduino Scenario Based
Programming

Scenario-1: Smart Street Light

- **Situation:** A street light should automatically turn ON at night and OFF during the day. If motion is detected at night, increase brightness.
- **Requirements:**
 - (a) LDR → detect day/night,
 - (b) PIR sensor → detect motion and
 - (c) LED (PWM) → control brightness
- **Programming Task:** Write an Arduino program that:
 - (a) Reads LDR value,
 - (b) Turns LED ON only at night and
 - (c) Increases brightness when motion is detected

Scenario-1: Smart Street Light - Solution

```
1 int ldrPin = A0;           // LDR connected to analog pin A0
2 int pirPin = 2;            // PIR sensor connected to digital pin 2
3 int ledPin = 9;            // LED connected to PWM pin 9
4
5 int ldrValue = 0;
6 int pirState = 0;
7
8 void setup() {
9   pinMode(pirPin, INPUT);
10  pinMode(ledPin, OUTPUT);
11  Serial.begin(9600);
12 }
13
14 void loop() {
15
16   ldrValue = analogRead(ldrPin); // Read LDR value
17   pirState = digitalRead(pirPin); // Read PIR sensor
18
19   Serial.print("LDR Value: ");
20   Serial.print(ldrValue);
21   Serial.print(" PIR: ");
22   Serial.println(pirState);
23
24   // Check for night condition
25   if (ldrValue < 500) { // Night time
26     if (pirState == HIGH) {
27       // Motion detected → High brightness
28       analogWrite(ledPin, 255);
29     }
30     else {
31       // No motion → Low brightness
32       analogWrite(ledPin, 80);
33     }
34   }
35   else {
36     // Day time → LED OFF
37     analogWrite(ledPin, 0);
38   }
39   delay(500); // Small delay for stability
40 }
```



Scenario-2 : Smart Parking System

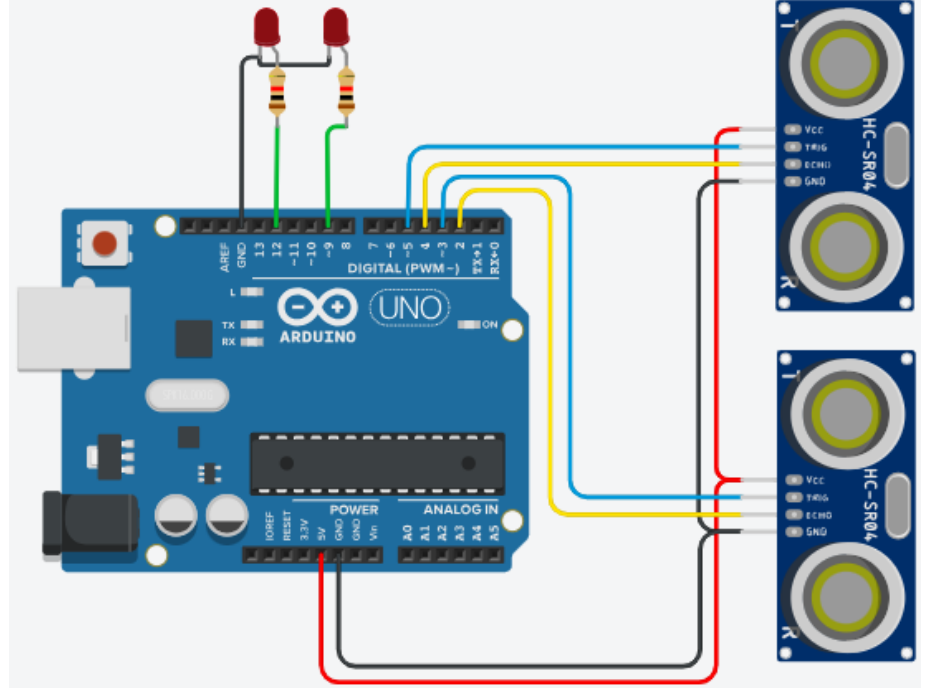
- **Scenario :** A smart parking system is required to monitor multiple parking slots automatically. Each parking slot should be able to detect the presence of a vehicle and indicate whether the slot is occupied or free. The system should also maintain and display the total number of available parking slots in real time.
- **Requirements**
 - (a) Ultrasonic sensors to detect vehicle presence in each parking slot
 - (b) LEDs to indicate slot status (occupied / free)
- **Programming Task :** Write program that performs the following tasks:
 - (a) Measures the distance using ultrasonic sensors for each parking slot
 - (b) Detects vehicle presence based on a predefined distance threshold
 - (c) Turns ON the corresponding LED when a parking slot is occupied
 - (d) Turns OFF the LED when a parking slot is free
 - (e) Counts the total number of available parking slots
 - (f) Displays the status of each slot and the total number of free slots on the Serial Monitor

Scenario-2 : Smart Parking System - Solution

```

1  #define slots 2
2
3  int trigPins[slots] = {3, 5};
4  int echoPins[slots] = {2, 4};
5  int ledPins[slots] = {9, 12};
6
7  long duration;
8  int distance;
9
10 void setup() {
11     Serial.begin(9600);
12
13     for (int i = 0; i < slots; i++) {
14         pinMode(trigPins[i], OUTPUT);
15         pinMode(echoPins[i], INPUT);
16         pinMode(ledPins[i], OUTPUT);
17     }
18 }
19
20 void loop() {
21     int freeSlots = 0;
22
23     Serial.println("---- Parking Status ----");
24
25     for (int i = 0; i < slots; i++) {
26         digitalWrite(trigPins[i], LOW);
27         delayMicroseconds(2);
28         digitalWrite(trigPins[i], HIGH);
29         delayMicroseconds(10);
30         digitalWrite(trigPins[i], LOW);
31
32         duration = pulseIn(echoPins[i], HIGH);
33         distance = duration * 0.034 / 2;
34
35         Serial.print("Slot ");
36         Serial.print(i + 1);
37         Serial.print(": Distance = ");
38         Serial.print(distance);
39         Serial.print(" cm → ");

```



```
40
41     if (distance < 15) {
42         digitalWrite(ledPins[i], HIGH);
43         Serial.println("OCCUPIED");
44     } else {
45         digitalWrite(ledPins[i], LOW);
46         Serial.println("FREE");
47         freeSlots++;
48     }
49 }
50
51 Serial.print("Total Free Slots: ");
52 Serial.println(freeSlots);
53 Serial.println("-----");
54
55 delay(3000);
56 }
```

Scenario-3 : Multi-Zone Temperature Monitoring

- **Scenario** : A smart building has three rooms, each monitored by a DHT11 temperature sensor. To avoid false alarms due to temporary temperature spikes in a single room, the system should raise an alert only when overheating is detected across multiple rooms for a sustained period.
- **Requirements**
 - (a) Use three DHT22 sensors, each connected to a different digital pin
 - (b) Read temperature values every 2 seconds
 - (c) Temperature threshold: 32 °C
 - (d) Trigger alert only if two or more rooms exceed the threshold for three consecutive readings
 - (e) Display output on Serial Monitor
- **Programming Task**: Write an Arduino program that:
 - (a) Reads temperature from all three DHT11 sensors
 - (b) Counts rooms exceeding the threshold
 - (c) Implements persistence logic using counters
 - (d) Displays “BUILDING OVERHEAT” or “NORMAL” accordingly

Scenario-3 : Multi-Zone Temperature Monitoring

```
1  #include <DHT.h>
2
3  DHT dht[] = { DHT(2, DHT22), DHT(3, DHT22), DHT(4, DHT22) };
4
5  int hotCount = 0;
6
7  void setup() {
8      Serial.begin(9600);
9      for (int i = 0; i < 3; i++)
10         dht[i].begin();
11 }
12
13 void loop() {
14     int hotRooms = 0;
15
16     for (int i = 0; i < 3; i++) {
17         float t = dht[i].readTemperature();
18         if (!isnan(t) && t > 32)
19             hotRooms++;
20     }
21
22     if (hotRooms >= 2)
23         hotCount++;
24     else
25         hotCount = 0;
26
27     if (hotCount >= 3)
28         Serial.println("BUILDING OVERHEAT");
29     else
30         Serial.println("NORMAL");
31
32     delay(2000);
33 }
```

